

The Semantic Layer for BI and AI: A Step-by-Step Guide to Trustworthy Natural-Language Analytics

A story in three diagrams, nine artifacts, and one number nobody in the room could defend.

The afternoon two numbers disagreed

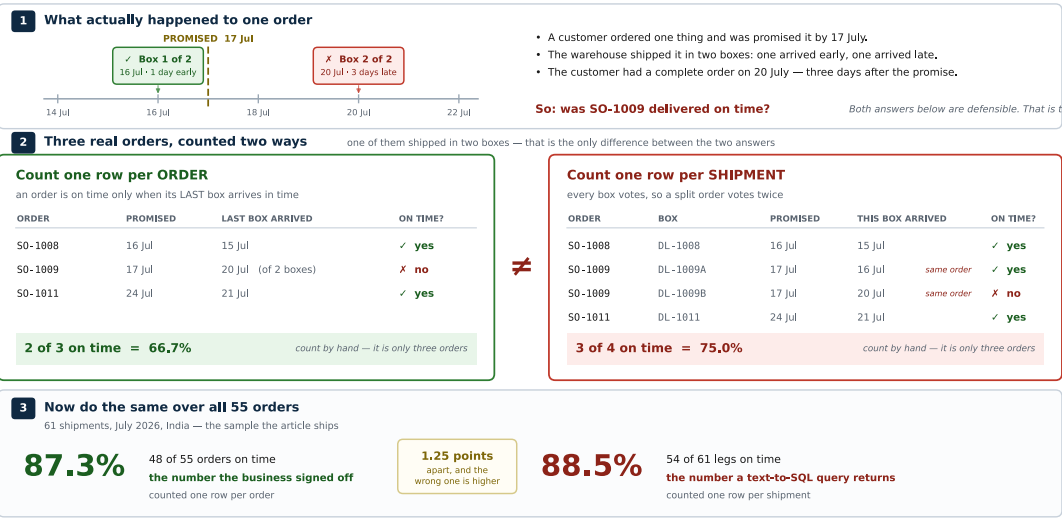
It was a quarterly review. The slide said on-time delivery for India last month was **88.5%**. Someone had re-run the same question that morning — *"on-time delivery for India warehouses last month"* — against the same DuckDB file, the same 132 orders, the same SQL dialect. They got **87.3%**.

Two numbers. One afternoon. One of them is wrong, and nobody in the room could say which.

That is the real problem, and it is worth being precise about why. Nothing crashed. No query threw an error. Both numbers are plausible, both round cleanly, and both have a SQL statement behind them that a competent engineer would sign off on. The disagreement is quiet, and quiet is exactly what makes it dangerous.

One order, two boxes, two different “correct” answers

Ask a computer “what was our on-time delivery last month?” in plain English and it can return either 87.3% or 88.5% from the same data. Here is where the two numbers come from.



One order that ships in two boxes can be counted as a single order or as two shipments. Count the shipments and you get 88.5%; count the orders and you get 87.3%. Full resolution: [docs/diagrams/grain_two_ways.svg](#) — scaled to fit the page here.

Where the wrong number comes from

Here is where 88.5% is hiding. The delivery table has one row per delivery *leg*. An order that ships in two legs — partial shipment on Tuesday, the rest on Friday — has two rows. Join orders to legs and count, and that order votes twice. Worse: if

the first leg arrived before the promised date, the order looks on time *twice over*, while the second leg that actually blew the promise is one row lost among sixty.

61 legs. 55 orders.

- Order grain: $48 / 55 = 87.27\%$
- Delivery grain: $54 / 61 = 88.52\%$

The gap is **1.25 points**. It is small enough to look like rounding, and it always errs *upward*. A wrong number that flatters you is the one that survives to the board deck, because nobody re-checks a figure that confirms the story they wanted to tell.

The model did not hallucinate. It wrote syntactically valid, semantically reasonable SQL against a table whose *grain* — what a single row is supposed to mean — nobody had written down anywhere it could read. **The failure was in the metadata, so the fix has to be in the metadata.**

The one decision everything else follows from

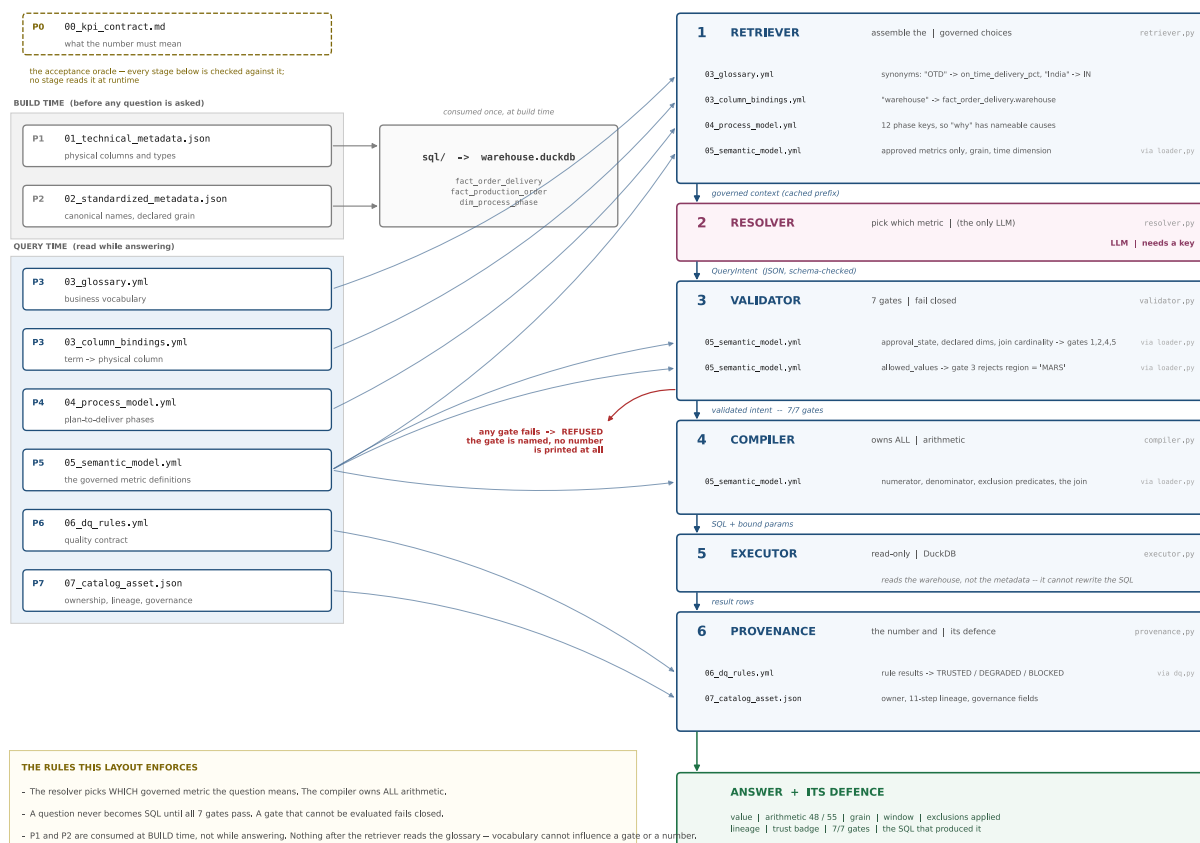
Everything below is a consequence of a single rule:

The model never writes SQL and never does arithmetic. It reads a governed metadata slice and emits a validated JSON object — which metric, which filters, which time window. A deterministic compiler turns that object into SQL.

This is the inversion that matters. Text-to-SQL asks a language model to be right about grain, joins, exclusions and fiscal calendars on *every single call*. This architecture asks it to be right about one much easier thing: **which of three approved metrics you meant**. Everything that has to be exactly right is compiled from metadata, by code, the same way every time. Two people asking the same question in different words get the same number — because the number was never in the model's hands.

How governed metadata answers a natural-language question

"Why did on-time delivery drop for India warehouses last month?" -> 87.3%, down 8.9pp, largest contributor: quality inspection



The pipeline is six stages: retriever → resolver → validator → compiler → executor → provenance. Only the resolver calls a model. Only the compiler writes SQL. Validation runs before compilation, so a refused question never becomes SQL at all. Full resolution: [docs/diagrams/nl_dataflow.svg](#).

The two rows worth a second look are the validator and the compiler: **neither ever opens a file**. Both receive a parsed `SemanticModel`, which means the compiler has no access to the glossary. Vocabulary is resolved *before* the gates, so which synonym a user happened to type can never reach the arithmetic.

Building it, one phase at a time

The method is one rule applied eight times: **each phase produces a real file and closes one specific way natural-language querying fails**. A phase that doesn't close a named failure is a phase you can skip.

Phase	Artifact	The NL failure it closes
P0	<code>00_kpi_contract.md</code>	Nobody agrees what "on-time" means, so no answer can be <i>wrong</i>
P1	<code>01_technical_metadata.json</code>	Column names harvested from SAP DDIC, with no business meaning
P2	<code>02_standardized_metadata.json</code>	Types and keys inconsistent across source systems
P3	<code>03_glossary.yml</code> + <code>03_column_bindings.yml</code>	"OTD", "delivery reliability", "promise adherence" reach no column
P4	<code>04_process_model.yml</code>	The model can answer <i>what</i> happened, never <i>why</i>
P5	<code>05_semantic_model.yml</code>	The model invents its own arithmetic — this is where 88.5% dies
P6	<code>06_dq_rules.yml</code>	A number from incomplete inputs is returned with full confidence
P7	<code>07_catalog_asset.json</code>	The answer arrives with no provenance, so nobody can defend it

Nine files, eight phases: P3 emits two, because a glossary term and its physical binding are different governance objects with different owners.

Two of those files carry most of the weight. The first is the glossary (P3), where vocabulary is pinned down in prose the model actually reads:

```
- term: On-Time Delivery %
  definition: >
    Share of eligible customer orders delivered on or before the promised
    delivery date. Measured at order grain (one row per order_id). An order
    counts as delivered only when its final delivery leg arrives. Excludes
    cancelled orders and orders not yet due.
  synonyms: [OTD, on time delivery, delivery reliability, promise adherence, OTIF]
  owner: VP Supply Chain
```

That one sentence — "*one row per order_id*" — is doing real work. But the load-bearing artifact is the governed metric itself (P5):

```

- name: on_time_delivery_pct
  label: On-Time Delivery %
  entity: fact_order_delivery
  grain: order
  expression:
    type: ratio_percent
    numerator: sum(CASE WHEN is_on_time THEN 1 ELSE 0 END)
    denominator: sum(CASE WHEN is_eligible THEN 1 ELSE 0 END)
  exclusions:
    - key: cancelled_orders
      predicate: order_status <> 'CANC'
      rationale: A cancelled order was never due; including it inflates the denominator.
  business_rules:
    - Order-grain only. Delivery-grain aggregation double-counts split shipments.
  owner: VP Supply Chain
  approval_state: approved

```

Four details, each one a defect that would otherwise ship: **grain: order** is *declared*, not inferred — this is the field that kills 88.5%. Exclusions carry a **rationale**, so nobody deletes them in six months assuming they were a mistake. **approval_state: approved** is read by the retriever, so an unapproved metric is never even offered to the model. And ratios are declared as **numerator and denominator**, never a pre-averaged rate — averaging percentages across groups of unequal size gives a different, wrong number that looks fine.

Seven gates that fail closed

Between the model's JSON and any SQL, seven checks run. All seven run every time, and a gate that cannot be evaluated **fails closed**:

Gate	Refuses
metric_approved	a metric that isn't approved, or doesn't exist
dimensions_declared	grouping by something the model invented
filters_bound	region = 'MARS' — a value outside the declared allowed set
grain_matches	the 88.5% query. It never becomes SQL.
no_fanout	a dimension only reachable through a one-to-many join
time_window_bounded	an unbounded or inverted date range
exclusions_applied	a metric whose certified exclusions can't be applied

Put the 88.5% query through the pipeline and this is the actual output — not a paraphrase:

```

REFUSED  on_time_delivery_pct: the intent did not pass validation.

gates failed  grain_matches
grain_matches    Metric 'on_time_delivery_pct' is certified at grain 'order'; the
                  intent asks for grain 'delivery'. Aggregating at the wrong grain
                  changes the number.

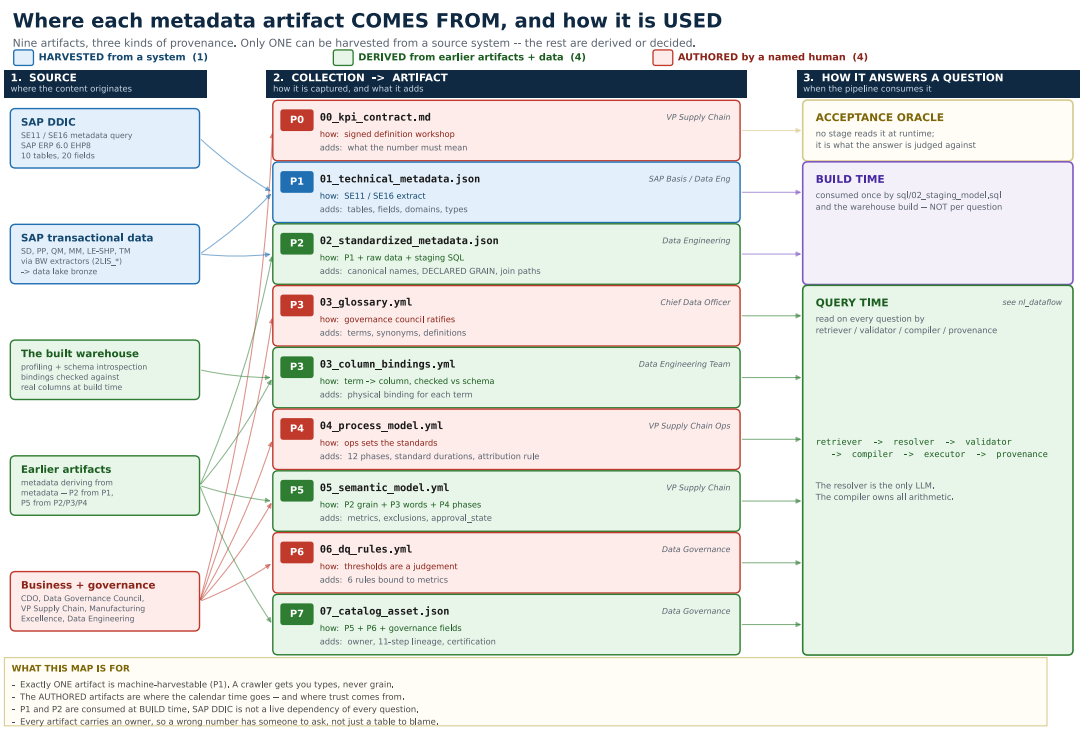
No number is shown. A value that failed a gate is not a caveated answer,
it is an unverified one.

```

"I can't answer that, and here is which rule stopped me" is a system you can put in front of a CFO. One that always produces a number is not.

"Can't we just harvest this from SAP?"

This is the question that lands next, usually from whoever is being asked to fund the work. The honest answer is a diagram and a count.



Read it left to right: source → how it was collected → the artifact → when the pipeline consumes it. The colours are the answer to the funding question. Full resolution: [docs/diagrams/metadata_sources.svg](#).

Exactly one of the nine artifacts is machine-harvestable. The other eight split two ways:

Provenance	Count	What it means
HARVESTED	1	a repeatable query against a running system (the SAP data dictionary)
DERIVED	4	computed from earlier artifacts plus the data
AUTHORED	4	a named human decided something no system knows

Stated as a planning fact: **a harvester gets you tables, fields, domains and types. It cannot get you grain, exclusions, an approved definition, or a standard duration** — and those are precisely the fields that decide whether an answer is *right* rather than merely well-formed. The 88.5% error lives in a field no crawler emits. This is why "point a catalog at the warehouse and turn on NL query" produces a demo, not a trustworthy system.

The question a dashboard can't answer

Descriptive questions — "*what was it?*" — need only a glossary, a grain and a metric. The question people actually ask is diagnostic:

"Why did on-time delivery drop for India warehouses last month?"

Answering it needs per-phase timestamps, per-phase planned baselines, and an attribution rule defined once, in metadata. This is why the manufacturing process is in scope: the answer to *why* lives upstream in production, not in the delivery table. Here is the real output:

ANSWER On-Time Delivery % is 87.3% -- 48 of 55 eligible orders delivered on or before the promised date

BREAKDOWN

Quality inspection	4
Transportation	2
Production execution	1
unattributed	0

Largest contributor: Quality inspection (4 of 7 late orders).

COMPARISON

prior window	June 2026 (2026-06-01 to 2026-06-30)
prior value	96.2% (25 / 26)
change	down 8.9pp (96.2% -> 87.3%)

On those four orders, the quality-management usage decision ran an average of **4.75 days over its 1.0-day standard, against 0.0 days of slack** — the promise date left no room to absorb it.

And here is the part that makes the case: **transportation ran under standard on 5 of the 7 late orders**. Logistics was the obvious suspect, and logistics was faster than planned. Without per-phase baselines you get a plausible, confident, wrong story — and you go optimize the wrong department. `unattributed 0` is printed even though it is zero, on purpose: an attribution that silently drops the orders it couldn't explain is one you can't audit.

What it honestly does not do

Governance metadata invites overreading, so this is worth stating plainly. Catalog metadata supports **discovery**. It is not policy enforcement, consent management, access control, or retention automation. An `owner` field records who is accountable; it does not stop anyone from querying anything. And the live resolver path — the one that calls a model over the network — is documented but **not** asserted as tested here; it needs API credentials this environment doesn't have, and the verification script reports that criterion as `SKIP` rather than quietly counting it as a pass. A piece arguing that overclaiming is the root problem cannot itself overclaim.

Reproduce every number above

Nothing here is illustrative. The guide's prose is under test alongside the code — every figure is asserted against the artifacts or the warehouse, because a reproduction guide whose numbers have drifted spends the reader's trust before it spends their time. Five commands, no API key, no network:

```
pip install -r requirements.txt
python src/build_warehouse.py
python src/ask.py --offline Q1      # 87.3%, 48/55, TRUSTED, 7/7 gates
python src/ask.py --offline Q2      # the diagnostic breakdown above
python verify_acceptance.py         # measures every acceptance criterion
```

The complete, tested project — sample data, all nine metadata artifacts, the pipeline, and a single self-contained file you can paste into a terminal — lives at github.com/BeenaKhandelwal/semantic-layer-nl.

The one-line version

Natural-language querying does not fail because models can't write SQL. It fails because the metadata that makes a question answerable *correctly* — grain, cardinality, exclusions, approved definitions, process baselines — was never written down where anything could read it. Build that, and the model's job shrinks to something it is genuinely good at.

What is your team's answer to "which of these two numbers is right?" If it's "ask Priya," you already have a semantic layer — it just isn't written down.